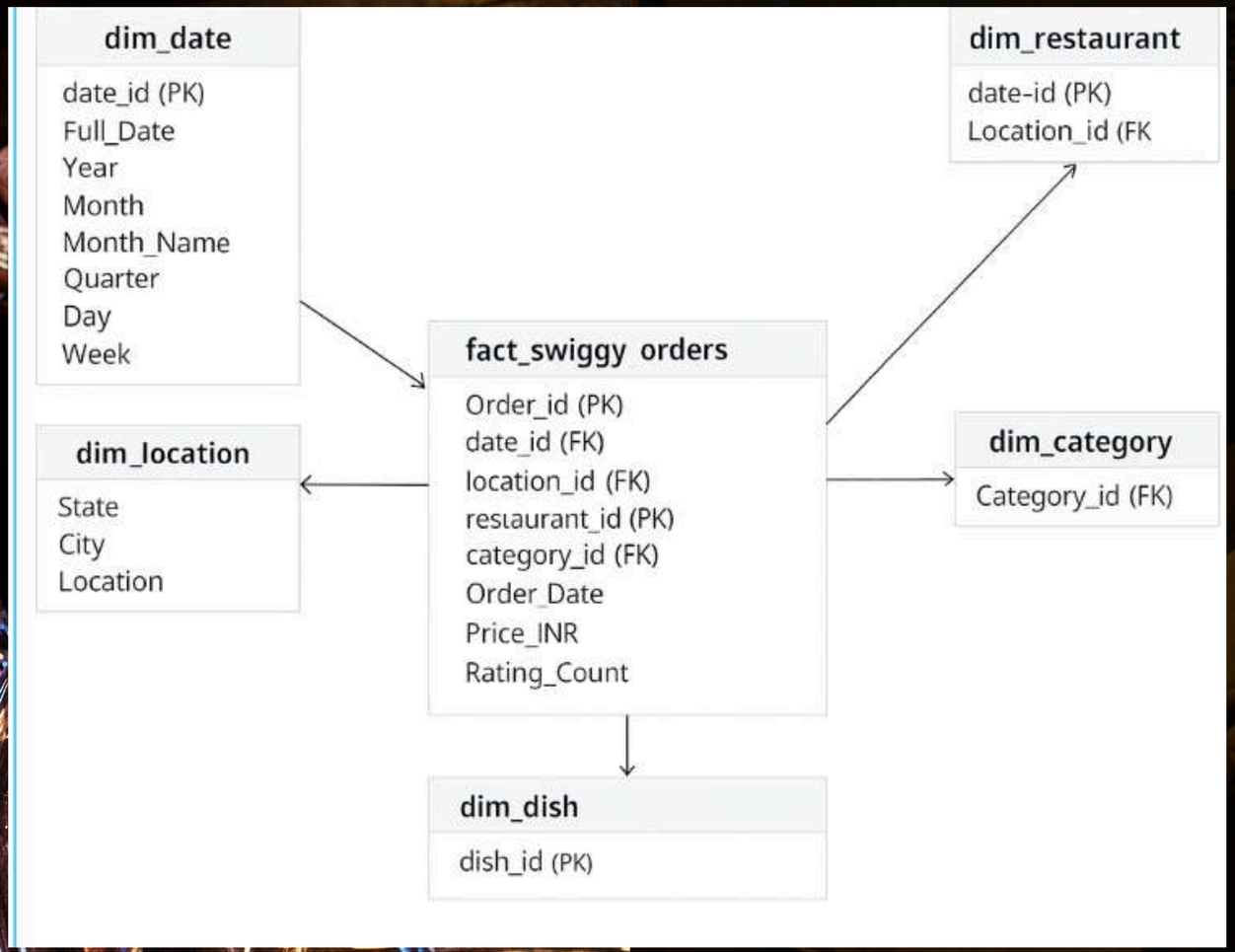
SQL SWIGGY SALES PROJECT

"PERFORMED END-TO-END SQL-BASED DATA CLEANING, MODELING, AND ANALYSIS ON SWIGGY ORDER DATA, UNCOVERING KEY INSIGHTS ON REVENUE, CUSTOMER BEHAVIOR, AND DEMAND TRENDS USING A STAR SCHEMA APPROACH."



"Designed a Star Schema data model for Swiggy order analytics, integrating fact and dimension tables (date, location, restaurant, category, and dish) to enable scalable SQL-based insights."

```sql
-- DATA CLEANING AND VALIDATION
-- NULL CHECK
SELECT
    SUM(CASE WHEN state IS NULL THEN 1 ELSE 0 END) AS null_state,
    SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS null_city,
    SUM(CASE WHEN order_date IS NULL THEN 1 ELSE 0 END) AS null_order_date,
    SUM(CASE WHEN restaurant_name IS NULL THEN 1 ELSE 0 END) AS null_restaurant_name,
    SUM(CASE WHEN location IS NULL THEN 1 ELSE 0 END) AS null_location,
    SUM(CASE WHEN category IS NULL THEN 1 ELSE 0 END) AS null_category,
    SUM(CASE WHEN dish_name IS NULL THEN 1 ELSE 0 END) AS null_dish_name,
    SUM(CASE WHEN price IS NULL THEN 1 ELSE 0 END) AS null_price,
    SUM(CASE WHEN rating IS NULL THEN 1 ELSE 0 END) AS null_rating,
    SUM(CASE WHEN rating_count IS NULL THEN 1 ELSE 0 END) AS null_rating_count
FROM swiggy_data;
```

"Implemented a NULL value audit using SQL to ensure data integrity before transformation and analysis."

```sql
-- EMPTY OR BLANK STRING
SELECT *
FROM swiggy_data
WHERE
state = '' OR city = '' OR restaurant_name = '' OR location = '' OR category = '' OR dish_name =
```

"SQL query to identify records with empty or blank string values across multiple columns for data validation."

```sql
-- DUPLICATE DETECTION
SELECT
state, city, order_date, restaurant_name, location, category, dish_name, price, rating, rating_count, count(*) AS CNT
FROM swiggy_data
GROUP BY state, city, order_date, restaurant_name, location, category, dish_name, price, rating, rating_count
HAVING count(*) > 1;
```

"Duplicate record identification using GROUP BY and HAVING clause to detect repeated entries in the dataset."

```sql
-- DELETE DUPLICATION

SELECT *
FROM (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY state, city, order_date, restaurant_name, location, category, dish_name, price, rating, rating_count
           ) AS rn
    FROM swiggy_data
) t
WHERE rn > 1;

CREATE TABLE swiggy_clean AS
SELECT DISTINCT *
FROM swiggy_data;

DROP TABLE swiggy_data;
RENAME TABLE swiggy_clean TO swiggy_data;
```

"Duplicate record detection and removal using ROW_NUMBER() window function and DISTINCT-based table reconstruction in SQL."

“Designed and implemented a Star Schema data model by creating dimension tables (date, location, restaurant, category, dish) and a central fact table with appropriate primary and foreign key relationships.”

```sql
-- CREATING SCHEMA
-- DIMENTION TABLE
-- DATA TABLE

CREATE TABLE dim_date (
    date_id INT PRIMARY KEY AUTO_INCREMENT,
    full_date DATE,
    year INT,
    month INT,
    month_name VARCHAR(20),
    quarter INT,
    day INT,
    week INT
);
```

```sql
-- DIM LOCATION
CREATE TABLE dim_location(
    location_id INT PRIMARY KEY AUTO_INCREMENT,
    state VARCHAR(100),
    city VARCHAR(100),
    location VARCHAR(200)
);
```

```sql
-- DIM CATEGORY
CREATE TABLE dim_category(
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category VARCHAR(200)
);

-- DIM DISH
CREATE TABLE dim_dish(
    dish_id INT PRIMARY KEY AUTO_INCREMENT,
    dish_name VARCHAR(200)
);
```

```sql
-- FACT TABLE
CREATE TABLE fact_swiggy_orders(
    order_id INT PRIMARY KEY AUTO_INCREMENT,

    date_id INT,
    price_inr DECIMAL(10,2),
    rating DECIMAL(4,2),
    rating_count INT,
    location_id INT,
    restaurant_id INT,
    category_id INT,
    dish_id INT,
    FOREIGN KEY (date_id) REFERENCES dim_date(date_id),
    FOREIGN KEY (location_id) REFERENCES dim_location(location_id),
    FOREIGN KEY (restaurant_id) REFERENCES dim_restaurant(restaurant_id),
    FOREIGN KEY (category_id) REFERENCES dim_category(category_id),
    FOREIGN KEY (dish_id) REFERENCES dim_dish(dish_id)
);
```

"TRANSFORMED RAW DATA INTO STRUCTURED DIMENSION TABLES BY INSERTING DISTINCT VALUES USING SQL, ENABLING EFFICIENT DATA MODELING AND ANALYSIS."

```sql
-- INSERT DATA INTO THE TABLE
-- dim data
INSERT INTO dim_date (full_date, `year`, `month`, month_name, quarter, `day`, week)
SELECT DISTINCT
    DATE(order_date),
    YEAR(order_date),
    MONTH(order_date),
    DATE_FORMAT(order_date, '%M'),
    QUARTER(order_date),
    DAY(order_date),
    WEEK(order_date)
FROM swiggy_data
WHERE order_date IS NOT NULL;
```

```sql
-- dim location
INSERT INTO dim_location (state, city, location)
SELECT DISTINCT
    state,
    city,
    location
 FROM swiggy_data;

 -- dim restaurant
INSERT INTO dim_restaurant (restaurant_name)
SELECT DISTINCT
    restaurant_name
FROM swiggy_data;
```

```sql
-- dim dish
INSERT INTO dim_dish(dish_name)
SELECT DISTINCT
    dish_name
FROM swiggy_data;

-- dim category
INSERT INTO dim_category(category)
SELECT DISTINCT
    category
FROM swiggy_data;
```

```sql
-- fact table
INSERT INTO fact_swiggy_orders (date_id, price_inr, rating, rating_count, location_id, restaurant_id, category_id, dish_id)
SELECT dd.date_id,
    s.price,
    s.rating,
    s.rating_count,

    dl.location_id,
    dr.restaurant_id,
    dc.category_id,
    dsh.dish_id
FROM swiggy_data s

JOIN dim_date dd
        ON dd.full_date = s.order_date
```

```sql
JOIN dim_date dd
        ON dd.full_date = s.order_date

JOIN dim_location dl
        ON dl.state = s.state
        AND dl.city = s.city
        AND dl.location = s.location

JOIN dim_restaurant dr
    ON dr.restaurant_name = s.restaurant_name

JOIN dim_category dc
    ON dc.category = s.category

JOIN dim_dish dsh
    ON dsh.dish_name = s.dish_name;
```

KEY PERFORMANENCE INDICATOR

```sql
-- KPI


-- TOTAL ORDERS
SELECT count(order_id) AS total_orders
FROM fact_swiggy_orders;
```

	total_orders
▶	3176

"Analyzed a dataset comprising 3,176 orders, forming the foundation for further KPI and performance analysis."

```sql
-- TOTAL REVENUE
SELECT
CONCAT(FORMAT(SUM(price_inr) / 100000, 2), ' INR Million') AS total_revenue
FROM fact_swiggy_orders;
```

Result Grid

total_revenue
6.54 INR Million

"Analyzed total revenue of ₹6.54 million across 3,176 orders, with an average order value of approximately ₹206, providing a baseline for further business insights."

```sql
-- AVERAGE DISH PRICE
SELECT
CONCAT(FORMAT(AVG(price_inr), 2), ' INR ') AS avg_dish_price
FROM fact_swiggy_orders;
```

Result Grid

	avg_dish_price
▶	205.86 INR

"The average order value is ₹205.86, suggesting a mid-range pricing model and consistent customer spending behavior across the dataset."

```sql
-- AVERAGE RATING
SELECT
FORMAT(AVG(rating),2) AS avg_rating
FROM fact_swiggy_orders;
```

Result Grid

	avg_rating
▶	4.13

"Achieved an average customer rating of 4.13, reflecting strong service quality and positive user experience across the dataset."

DEEP-DIVE INTO BUSINESS ANALYSIS

```sql
-- MONTLY ORDERS TREND
SELECT
d.year,
d.month,
d.month_name,
COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_date d
ON  f.date_id = d.date_id
GROUP BY d.year,
d.month,
d.month_name
ORDER BY COUNT(*) DESC;
```

Result Grid | Filter Rows:

year	month	month_name	total_orders
2025	2	February	644
2025	8	August	460
2025	5	May	452
2025	4	April	428
2025	7	July	388
2025	1	January	372
2025	6	June	368
2025	3	March	64

"Monthly order trend analysis revealed peak demand in February (644 orders), followed by an unusual drop in March, and stable recovery in subsequent months—highlighting both growth opportunities and potential data anomalies.

```sql
-- Quarterly order trends
SELECT d.year,
d.quarter,
COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_date d
ON f.date_id = d.date_id
GROUP BY d.year, d.quarter
ORDER BY d.quarter;
```

Result Grid | Filter Rows:

year	quarter	total_orders
2025	1	1080
2025	2	1248
2025	3	848

"Quarterly analysis revealed peak performance in Q2 (1248 orders) with ~15% growth from Q1, followed by a significant drop in Q3, highlighting seasonal demand fluctuations and optimization opportunities."

```sql
-- Day-of-week patterns
SELECT d.year, d.week,
d.day,
COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_date d
ON f.date_id = d.date_id
GROUP BY  d.year,d.week, d.day
ORDER BY d.year, d.week;
```

Result Grid | Filter Rows:

year	week	day	total_orders
2025	0	1	12
2025	0	2	20
2025	0	3	8
2025	0	4	12
2025	1	5	12
2025	1	6	8

"Analyzed order distribution across days, identifying fluctuations in daily demand and highlighting opportunities for targeted promotions on low-performing days."

```sql
-- Top 10 restaurants by orders
SELECT r.restaurant_name,
SUM(f.price_inr) AS total_revenue
FROM fact_swiggy_orders f
JOIN dim_restaurant r
ON r.restaurant_id = f.restaurant_id
GROUP BY r.restaurant_name
ORDER BY SUM(f.price_inr) DESC;
```

Result Grid | Filter Rows:

restaurant_name	total_revenue
Olio - The Wood Fired Pizzeria	151300.00
Srinidhi Sagar Deluxe	148604.00
Pizza Hut	99408.36
Nisarga Family Daba	76012.00
Sri Vinayaka Hot Chips	34120.00
Chinese Wok	33572.00
Appu Donne Biriyani Palace	29952.00
Big Bowl	28928.00
A2B - Adyar Ananda Bhavan	20560.00
Swadh Restaurant	15780.00
Thalassery Restaurant	12380.00

"Revenue analysis revealed that Olio – The Wood Fired Pizzeria leads with ₹151K, followed closely by Srinidhi Sagar Deluxe, with a sharp drop after the top performers—indicating revenue concentration among a few restaurants."

```sql
-- Top categories (Indian, Chinese, etc.)
SELECT c.category,
COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_category c
ON c.category_id = f.category_id
GROUP BY c.category
ORDER BY COUNT(*) DESC;
```

Result Grid | Filter Rows:

category	total_orders
Recommended	544
Snacks	256
North Indian Gravy	136
Non Veg Starter	120
Drinks	72
Manchurian & Chilly	64
Fresh Fruit Juice	64
Tandoori Breads	60
Dosas	60
Chaats	56
Breakfast Combos	56

"Category analysis shows 'Recommended' items lead with 544 orders, followed by Snacks, highlighting the impact of platform recommendations and strong demand for quick-consumption food."

```sql
-- Most ordered dishes
SELECT d.dish_name,
COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_dish d
ON d.dish_id = f.dish_id
GROUP BY d.dish_name
ORDER BY COUNT(*) DESC;
```

Result Grid | Filter Rows:

dish_name	total_orders
Garlic Naan	12
Curd Rice	12
Mushroom Masala	12
Butter Roti	12
Boiled egg	12
Badam Milk	8
Mutton Pepper Dry	8
Chow Chow Bath	8
Kesari Bath	8
Mix Raitha	8
Srinidhi Sagar Spe...	8

"Dish-level analysis shows no single dominant item, with multiple dishes tied at the top, indicating diverse customer preferences and consistent demand for staple food items."

```sql
--  Cuisine performance → Orders + Avg Rating
SELECT
c.category,
COUNT(*) AS total_orders,
AVG(CAST(f.rating AS DECIMAL(3,2))) AS avg_rating
FROM fact_swiggy_orders f
JOIN dim_category c
ON f.category_id = c.category_id
GROUP BY c.category
ORDER BY total_orders DESC;
```

Result Grid | Filter Rows:

category	total_orders	avg_rating
Recommended	544	4.283088
Snacks	256	4.000000
North I Non Veg	136	4.047059
Non Veg Starter	120	3.993333
Drinks	72	4.383333
Manchurian & Chilly	64	4.093750
Fresh Fruit Juice	64	4.168750
Tandoori Breads	60	4.173333
Dosas	60	4.093333
Chaats	56	4.007143
Breakfast Combos	56	4.000000

"Cuisine analysis revealed that 'Recommended' items drive both high demand and strong ratings, while categories like Drinks show high satisfaction but lower order volume—highlighting opportunities for targeted promotion."

```sql
-- Distribution of dish ratings from 1-5.
SELECT rating,
COUNT(*) AS rating_count
FROM fact_swiggy_orders
GROUP BY rating
ORDER BY rating;
```

Result Grid | Filter R

rating	rating_count
3.90	88
4.00	1412
4.10	100
4.20	188
4.30	340
4.40	168
4.50	120
4.60	108
4.70	124
4.80	84
4.90	60

"Rating distribution analysis shows a strong concentration around 4.0, indicating consistent customer satisfaction, with fewer high-end ratings suggesting opportunities to enhance customer experience."